Тема 19: Управление на качеството на софтуерни приложения. Тестване на софтуер.

ТЕОРИЯ

1. Осигуряване на качеството

Качество на софтуера = степента, до която система/компонент/процес отговаря на **специфицираните изисквания** и удовлетворява **нуждите на заинтересованите лица**.

Осигуряване на качеството (QA) = набор от дейности (вкл. тестване), проектирани да:

- Докажат правилното поведение на продукта
- Установят и отстранят проблеми

Алтернативи за осигуряване на качество:

- **Процесно-ориентирани** → QA на разработката (контрол на процеса)
- **Продуктово-ориентирани** → QA на поддръжката (контрол на продукта)

---

2. Тестови дейности, управление и автоматизация

Основни тестови дейности

Дейност	Описание
Планиране	Дефиниране на тестова цел; планиране на ресурси, персонал; избор на техники
Конструиране на тестов модел	Идентифициране на източници; анализ и създаване на модел
Генериране на тестови сценарии	Създаване на Test Cases от модела
Управление на тестови пакети	Организиране на Test Suites
Подготовка на тестова процедура	Последователност и превключване на тестовите серии
Изпълнение и наблюдение	Заделяне на ресурси; изпълнение; проверка с тестови оракули
Анализ и проследяване	Анализ на резултати; идентифициране на повреди

Ключови понятия:

Термин	Дефиниция
Test Case (тестови сценарий)	Колекция от елементи + свързана информация за изпълнение на един тест
Test Run (тестова серия)	Динамично изпълнение на специфични тестови дейности

Test Suite (тестов пакет)	Колекция от тестови сценарии в последователност до критерий за спиране

Управление на тестовия екип

Модел	Описание
Вертикален	Около продукта; задачи асоциирани с изпълнители
Хоризонтален	Един тип тестване за всички продукти (за големи организации)
Смесен	Ниски нива — в проектите; системно тестване — споделено; централна поддръжка

Автоматизация

Цел: Освобождаване от повтарящи се задачи + повишаване на тестова производителност.

Важни въпроси:

- Кои сценарии могат да се автоматизират?
- Кои инструменти са подходящи?
- Каква е крайната цена?

---

3. Видове тестване

3.1 Тестване с контролни списъци (Checklist-Based)

Ad-hoc тестване = стартиране на софтуера, наблюдение, идентифициране на проблеми.
Checklist тестване = неформален TODO списък за проследяване на изтестваните елементи.

Тип	Описание
Базови	Прости списъци от елементи
Йерархични	Многонивови (горните съдържат долни)
Комбинирани (многомерни)	Всеки списък се обхожда за всички елементи от останалите
Смесени	Комбинация от йерархични и комбинирани

Ограничения: Трудно пълно покритие; припокриване между списъци; сложни взаимодействия.

3.2 Тестване с класове на еквивалентност (Equivalence Partitioning)

Идея: Входният домейн се разделя на класове, при които всички стойности се обработват **по еднакъв начин**. Избираме по 1 тест от всеки клас → пълно покритие с минимум тестове.

Стъпки:

1. Дефиниране на класове на еквивалентност (валидни и **невалидни**)
2. Избор на 1 тестови сценарий за всеки клас
3. Постигане на пълно покритие

Типове разделяне:

- Базирано на **софтуерни елементи** (полета, бутони)
- Базирано на **свойства, релации, логически условия**
- Комбинирано

Пример: Поле "Възраст" с валидни стойности 0–120:

- Клас 1 (невалиден): < 0 → тест: -1
- Клас 2 (валиден): 0–120 → тест: 50
- Клас 3 (невалиден): > 120 → тест: 150

3.3 Разделяне на входния домейн и тестване на границите (Boundary Value Analysis)

Идея: Грешките се концентрират около **граничните стойности** на класовете.

Стъпки:

1. Идентифициране на входното пространство + дефиниране на домейн
2. Разделяне на поддомейни
3. Анализ на границите по всички измерения
4. Избор на тестови точки: граница - 1, на границата, граница + 1
5. Тестване и анализ

Гранични тестови точки (за домейн [a, b]):

- `a - 1` (точно под долната граница)
- `a` (долна граница)
- `a + 1` (точно над долната)
- `b - 1` (точно под горната)
- `b` (горна граница)
- `b + 1` (точно над горната)

Проблеми при определяне на граници:

- Грешка на затвореността (отворена граница → затворена)
- Изместване на границата
- Липсваща граница
- Излишна граница

Пример: Поле "Оценка" 2.00–6.00:

Тест	Стойност	Очакван резултат
Под долна	1.99	Грешка
Долна граница	2.00	Валидна
Над долна	2.01	Валидна
Под горна	5.99	Валидна
Горна граница	6.00	Валидна
Над горна	6.01	Грешка

3.4 Други техники за тестване

Техника	Описание
Покритие на класове (CFG)	Граф на контролния поток; покритие на пътища/ребра/възли
Машина на крайните състояния	Тестване на преходи и действия в state machine
Граф на данновия поток	Проследяване на дефиниции и употреба на промен.
Таблица за вземане на решение	Комбинации от условия и действия в таблица
Тестване по двойки (Pairwise/Orthogonal)	Покриване на всяка двойка стойности поне веднъж
Разделяне на входния домейн	Комбинации от поддомейни

---

4. Нива на тестване (Testing Pyramid)



Ниво	Метод	Изпълнява се от	Описание
Unit Testing	Бяла кутия	Разработчиците	Функция, метод или клас изолирано
Integration Testing	Бяла + Черна	QA/Dev	Взаимодействие между модули
System Testing	Черна кутия	QA	Цялата система като едно цяло
Acceptance Testing	Черна кутия	Клиент/QA	Готовност за доставка; в Scrum — в края на Sprint

Бяла кутия (White-box): Тестерът познава вътрешната реализация.
Черна кутия (Black-box): Тестерът вижда само I/O; не познава вътрешността.

Допълнителни видове:

- **Beta Testing** — тестване от реални потребители в real environment
- **Regression Testing** — проверка, че нови промени не са счупили съществуващо
- **Performance Testing** — бързодействие под очаквано натоварване
- **Load Testing** — поведение под максимално натоварване

---

5. Метрики — измерване и класификация

Измерване = процес за съпоставяне на стойности на характеристики по определени правила.

Термин	Значение
Мярка	Конкретна стойност от измерването
Метрика	Функция, съпоставяща характеристиките на мерките им

Характеристики на добра метрика

Характеристика	Описание
Надеждна (Еднозначна)	Еднакви резултати при еднакви условия
Уместна	Измерва значим атрибут
Валидна	Измерва точно искания атрибут
Неподатлива	Не се влияе от предубедени намеси
Взаимно изключваща	Не дублира друга метрика
Сравнима	Може да се сравнява с други измервания/стандарти
Практична	Лесна за събиране и анализ

Класификация на метриките

Вид	Описание	Примери
Процесни	Ефективност на разработката	Брой дефекти на етап; % задачи в срок
Продуктови	Качества на продукта	MTTF, Defect rate, Customer satisfaction
Проектни	Управление на проекта	Цена, Ресурси, Срокове

Метрики за процес

- **Брой открити дефекти на етап** (колко грешки намираме по фази)
- **Средно време за изпълнение на задачи**
- **% завършени задачи в срок**
- **Брой завършени задачи**

Метрики за продукт

Метрика	Описание
MTTF (Mean Time To Failure)	Средно време до отказ
MTTR (Mean Time To Repair)	Средно време за възстановяване след отказ
Defect Rate	Брой/% дефекти в продукта
Customer Problems	Брой проблеми идентифицирани от потребителите

Customer Satisfaction	Удовлетвореност (анкети, NPS)
Време за обучение	Колко бързо нов потребител овладява системата
Използване на ресурси	CPU, RAM при нормално натоварване

КАК СЕ РЕШАВАТ ЗАДАЧИ

> ***Task 4*** *на практическия изпит (2021–2024): Дефиниране на тестови сценарии с конкретна техника.*

Тип 1: Класове на еквивалентност (Equivalence Partitioning)

Задача: Функция `discount(price)` дава 10% намаление ако 100 ≤ price ≤ 1000, 0% иначе.

Стъпка 1 — Дефинирай класовете:

Клас	Диапазон	Валиден?	Представителна стойност
EC1	price < 100	Невалиден	50
EC2	100 ≤ price ≤ 1000	Валиден	500
EC3	price > 1000	Невалиден	2000

Стъпка 2 — Тестови сценарии (1 на клас):

ID	Вход	Очакван резултат
TC1	price = 50	0% намаление
TC2	price = 500	10% намаление
TC3	price = 2000	0% намаление

Тип 2: Гранично тестване (Boundary Value Analysis)

Задача: Същата функция. Граници: 100 и 1000.

Тестови точки (On/Off point + за всяка граница):

ID	Вход	Описание	Очакван резултат
TC1	99	Под долна граница	0%
TC2	100	Долна граница (вкл.)	10%
TC3	101	Над долна граница	10%
TC4	999	Под горна граница	10%
TC5	1000	Горна граница (вкл.)	10%
TC6	1001	Над горна граница	0%

Тип 3: Таблица за вземане на решение (Decision Table)

Задача: Система предлага отстъпка: Member = Да/Не, Purchase > 500 = Да/Не.

Условия (Conditions):

	Правило 1	Правило 2	Правило 3	Правило 4
Member	Д	Д	Н	Н
Purchase > 500	Д	Н	Д	Н
Отстъпка 20%	✓	×	×	×
Отстъпка 10%	×	✓	✓	×
Без отстъпка	×	×	×	✓

4 правила → 4 тестови сценария.

Тип 4: Тестване по двойки (Pairwise)

Задача: 3 параметъра, всеки с 2 стойности: A (a1/a2), B (b1/b2), C (c1/c2).

Всяка комбинация в пълен тест: $2^3 = 8$. С pairwise покриваме всяка *двойка* стойности поне веднъж.

Минимален pairwise набор:

Тест	A	B	C
TC1	a1	b1	c1
TC2	a1	b2	c2
TC3	a2	b1	c2
TC4	a2	b2	c1

→ 4 теста вместо 8, но всяка двойка е покрита.

Тип 5: State-Based Testing (Машина на крайните състояния)

Задача: Поръчка може да е в: New → Confirmed → Shipped → Delivered. Незаплатена поръчка може да се анулира.

```
[New] --confirm()--> [Confirmed] --ship()--> [Shipped] --deliver()--> [Delivered]
[New] --cancel()--> [Cancelled]
[Confirmed] --cancel()--> [Cancelled]
```

Тестови сценарии = покриване на всички преходи:

ID	Начало	Действие	Очаквано крайно
TC1	New	confirm()	Confirmed

TC2	Confirmed	ship()	Shipped
TC3	Shipped	deliver()	Delivered
TC4	New	cancel()	Cancelled
TC5	Confirmed	cancel()	Cancelled

Бърза справка

Техника	Кога	Брой тестове
Checklist	Ad-hoc; бързо	Произволно
Equivalence Partitioning	Входен домейн	1 на клас
Boundary Value Analysis	Граници	6 на граница (min-1, min, min+1, max-1, max, max+1)
Decision Table	Комбинации условия	2^n (n условия)
Pairwise	Много параметри	<< 2^n
State Machine	Преходи	= брой преходи
Unit Test	Функция/метод	Бяла кутия
Integration	Модули	Черна/Бяла
System	Цяла система	Черна кутия
Acceptance	Доставка	Черна кутия
MTTF	Продуктова метрика	Средно вр. до отказ
MTTR	Продуктова метрика	Средно вр. за ремонт